System Modeling with UML

Context · Use Case · Sequence · Class · Activity · State Machine Diagrams

By. Soon Phei Tin

Based on Sommerville, Software Engineering (9th ed.) and Fowler, UML Distilled (3rd ed.)

AGENDA

What We'll Cover Today

1 · Foundations

Why we model, four perspectives on a system, and how precise a model needs to be.

2-3 · Interaction

Context models, use case diagrams and sequence diagrams — how a system meets the outside world.

4 · Structure

Class diagrams: notation, visibility, signatures, composition vs. aggregation, and identifying classes.

5-6 · Behaviour

Activity diagrams with swimlanes and advanced notation; state machines with composite states.

1

What Is System Modeling?

Why engineers draw pictures of systems before they build them

Why We Model

- Real systems are huge — nobody can hold all the detail in their head at once.
- Most failed projects fail for three ordinary reasons: misunderstood requirements, problems found too late, or excess complexity.
- A model deliberately leaves out detail so one aspect — context, structure, behaviour — is easy to discuss.
- **Since the 1990s, UML has been the standard notation family for these pictures.**

Existing System vs. Planned System

Existing system

A requirements-engineering activity. Clarifies what the current system does and gives a shared basis for discussion.

Planned (new) system

A design activity. Explains a proposed design, argues about alternatives, and documents it well enough to build.

Model-driven engineering

The model isn't just documentation — some tool chains generate an implementation directly from it.

Four Perspectives on One System

Perspective	The question it answers	Diagrams used
External / context	What lies outside my system?	Context diagrams
Interaction	How do people and systems interact?	Use case, sequence
Structural	How is the system's data structured?	Class diagrams
Behavioural	How does the system change over time?	Activity, state machine

How Precise Does a Model Need to Be?

- As a discussion tool — incomplete or slightly incorrect is fine; its job is to support conversation.
- As documentation of an existing system — must be accurate, but can still leave out irrelevant parts.
- As a specification for code generation — must be correct and complete; a machine acts on it directly.
- **Before you draw anything: who is this diagram for, and what decision are they trying to reach?**

2

Context Models

What lies outside the system we're building?

Reading a Context Diagram

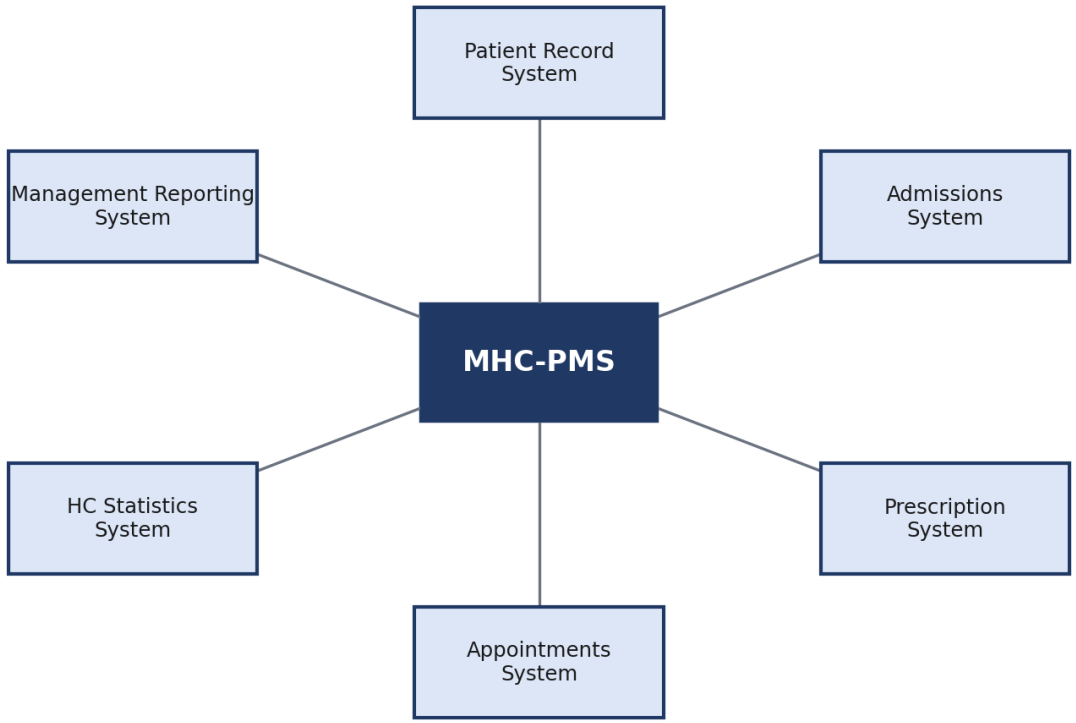


Figure — Context model for the MHC-PMS
(shows what lies outside the system boundary, not how it is used)

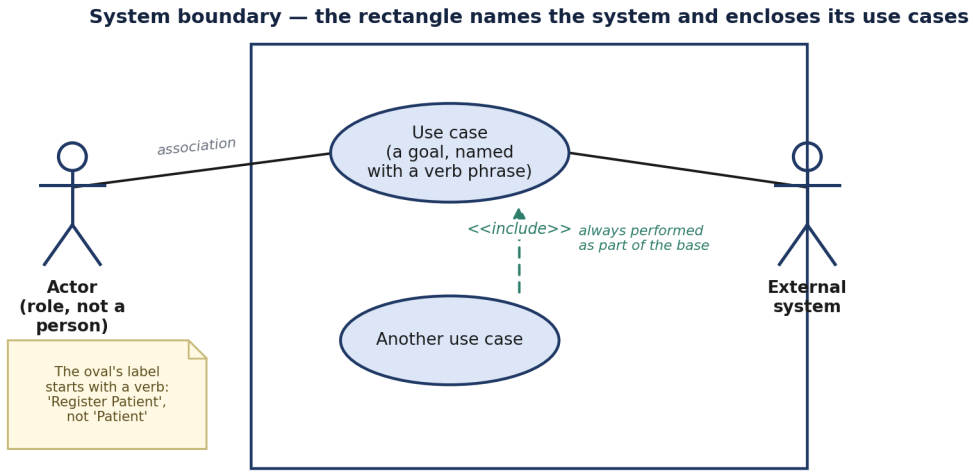
- Draws a boundary and shows only the other systems it touches.
- Says nothing about how the system is used, or what's inside it.
- Where the boundary sits is “a political judgment” — not purely technical.
- Usually paired with an activity diagram of the surrounding process.

3

Interaction Models I

Use Case Diagrams

The Anatomy of a Use Case Diagram



A **use case diagram** is a high-level picture of what a system does and who interacts with it.

It captures the system's functional requirements from the user's point of view, without showing internal design, data structures, or the order in which things happen.

A **use case** in a use case diagram is a specific goal that an actor achieves by interacting with the system.

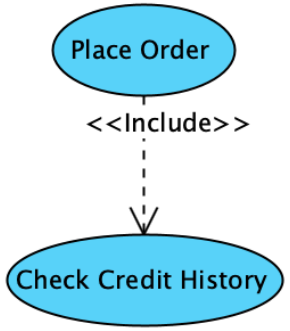
- System boundary — rectangle naming the system.
- Actor — a role, not a person; a stick figure outside the boundary.
- Use case — an oval named with a verb phrase.
- Association — a plain line, actor to use case.

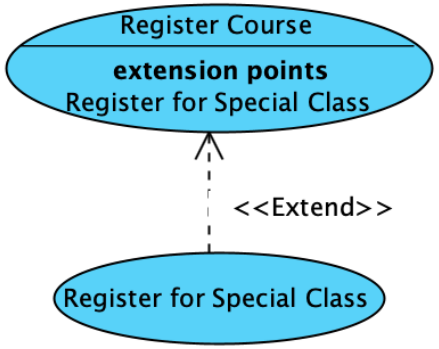
Actors, in More Depth

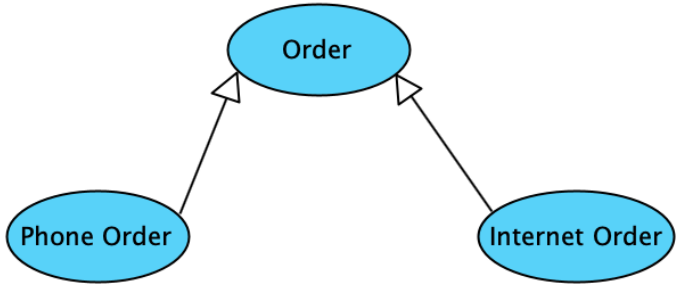
- An actor is anything external that produces or consumes data at the boundary — person, system, device, even a timer.
- Primary — a user whose goal the use case exists to fulfil.
- Supporting — provides a service to the use case (e.g. a payment processor).
- Offstage — has a stake in the outcome without using the system directly (e.g. a regulator).

Include, Extend & Generalization

- «include» — always happens; arrow points to the included use case.
- «extend» — optional; arrow points back to the base use case.
- Generalization — a child use case inherits a parent's behaviour.
- Rule of thumb: include is “I always need this”; extend is “I might add this.”







A Checklist for a Clean Diagram

- Identify the boundary first, and name it clearly.
- Show actors as roles, outside the boundary.
- Name use cases as goals, with a verb phrase.
- «include» mandatory behaviour; «extend» optional behaviour.
- Use generalization sparingly.

Worked Example: Requirements First, Then the Diagram

- A real requirements engineer starts from what the customer says, not from a blank canvas.
- EARS (Easy Approach to Requirements Syntax) writes that down in a few fixed sentence shapes, so nothing stays ambiguous.
- **The rest of this worked example — diagram, fully-dressed description, sequence diagram — is all derived from the same requirements set.**

Requirements First: EARS Format

Pattern	Shape
Ubiquitous	The <system> shall <response>.
Event-driven	WHEN <trigger>, the <system> shall <response>.
State-driven	WHILE <state>, the <system> shall <response>.
Unwanted behaviour	IF <condition>, THEN the <system> shall <response>.
Complex	A combination, e.g. WHILE <state> IF <condition> THEN the <system> shall <response>.

Requirements Collected from the Customer (excerpt)

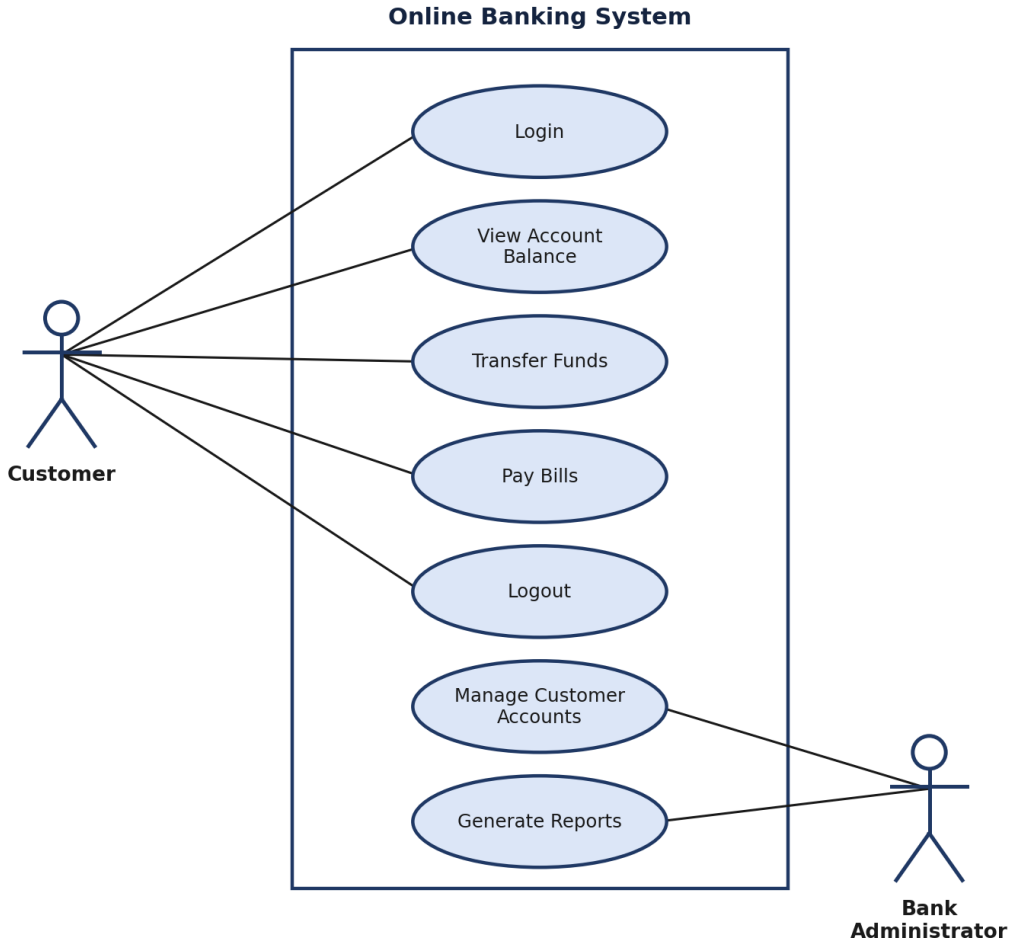
- OBS-01 (Ubiquitous): The system shall require login before any account function.
- OBS-06 (Event-driven): WHEN a customer selects “Transfer Funds”, the system shall display eligible accounts.
- OBS-08 (Unwanted): IF the amount exceeds the balance, THEN the system shall show insufficient-funds and stop.
- OBS-14 (State-driven): WHILE the confirmation screen is shown, the system shall allow cancel instead of confirm.

Full set (OBS-01 – OBS-20) is in the use case description example PDF.

From Requirements to the Diagram

Use Case	Actor	Source Requirements
Login	Customer	OBS-01, 02, 03, 04
View Account Balance	Customer	OBS-05
Transfer Funds	Customer	OBS-06 – 14
Pay Bills	Customer	OBS-15
Logout	Customer	OBS-16, 17
Manage Customer Accounts	Bank Administrator	OBS-18, 19
Generate Reports	Bank Administrator	OBS-18, 20

Worked Example — Online Banking System



Reconstructed directly from the EARS requirements above, not invented from scratch.

Use Case Description

Check out the use case description example PDF

Field	What it captures
Use case / Primary actor	The use case's name, and the actor whose goal drives it.
Preconditions	What must already be true before the use case can start.
Main flow	The numbered happy-path steps, in order.
Alternate flow(s)	Labelled by the step they branch from (e.g. “4a”) — a different, still-valid path.
Exception flow(s)	Labelled the same way — caused by a fault, not a user choice.
Postconditions	What must be true once the use case ends, for both success and failure.

Worked Example — Transfer Funds: Main Flow

1. Customer selects “Transfer Funds” from the account menu. [OBS-06]
2. System displays the customer's eligible accounts. [OBS-06]
3. Customer selects a source account, destination account, and amount. [OBS-07]
4. System validates the amount and the destination account. [OBS-07]
5. System displays a confirmation screen. [OBS-10]
6. Customer confirms the transfer. [OBS-11]
7. System debits, credits, and generates a reference number. [OBS-11]
8. System displays a transfer receipt and records the transaction. [OBS-13]

Transfer Funds — Alternate & Exception Flows

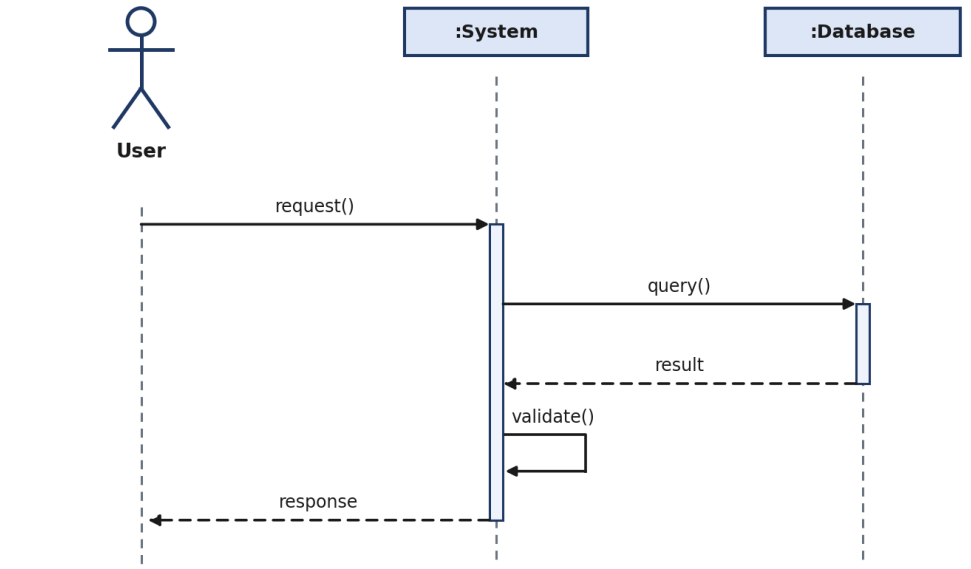
Ref	At step	Condition	System response
4a	4	Insufficient funds.	Insufficient-funds message; return to step 3. [OBS-08]
4b	4	Invalid destination account.	Invalid-account message; return to step 3. [OBS-09]
6a	6	Customer cancels.	Discard transfer; return to Account Summary. [OBS-14]
7a (exception)	7	System/network fault while posting.	Roll back; transfer-failed message; keep reference. [OBS-12]

4

Interaction Models II

Sequence Diagrams

The Anatomy of a Sequence Diagram



Reading a sequence diagram — vertical = time (top to bottom), horizontal position = which object is acting:

- solid arrow, filled head — synchronous call (caller waits)
- dashed arrow — return message
- thin vertical bar — activation (object executing / on call stack)
- self-loop arrow — the object calls its own method

A sequence diagram shows how parts of a system communicate and work together over time in sequence to accomplish one specific task.

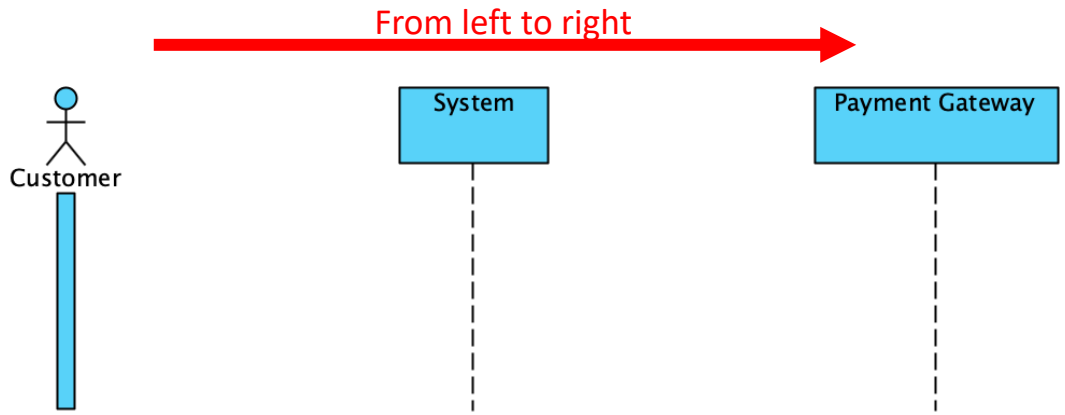
- Participants — shown as boxes along the top. An actor such as a human role or an external system, drawn as a stick figure. An object or component, drawn as a rectangle
- Lifeline — a participant's timeline; time runs downward.
- Activation bar — participant actively executing or waiting.
- Synchronous message — show how participants communicate. Solid arrow, filled head, caller waits.
- Return message — dashed arrow, response to synchronous message.

Grouping Behaviour with Frames/Fragments

- alt — alternatives (if/else); each branch carries its own guard.
- opt — an optional block; runs only if its one guard holds.
- loop — the enclosed messages repeat per the guard condition.

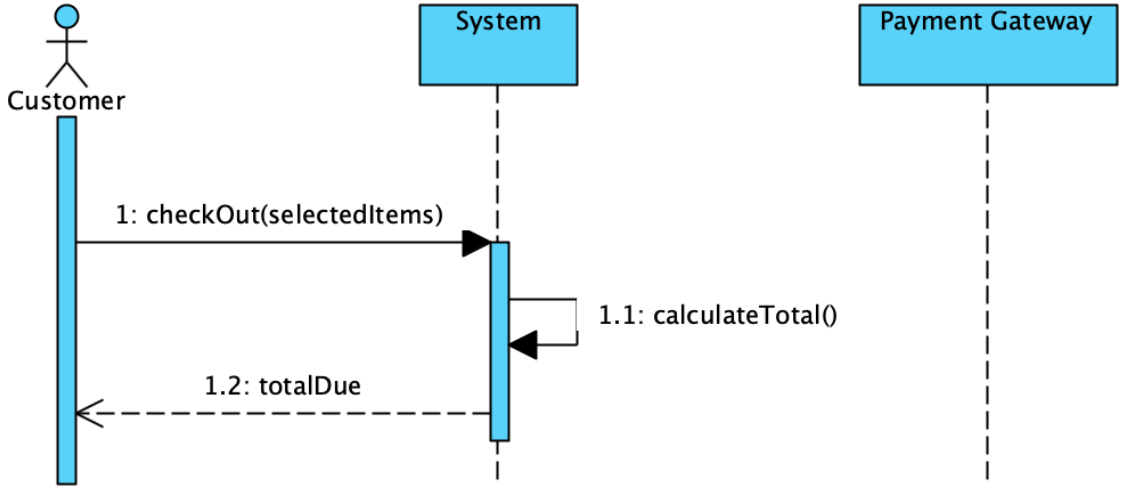
Drawing a Sequence Diagram

- Pick one scenario - Name it concretely, like "user places an order successfully" or "payment is declined". Draw separate diagrams for the happy path and for important error cases, if draw them in one diagram becoming complex.
- Decide level of detail - Keep the level consistent across the diagram. Mixing "Payment Gateway" with "StringUtils" in one picture makes it hard to read.
- List the participants - Write down everyone involved in the scenario and place them left to right, roughly in the order they first take part. The initiator, usually an actor, goes on the far left.



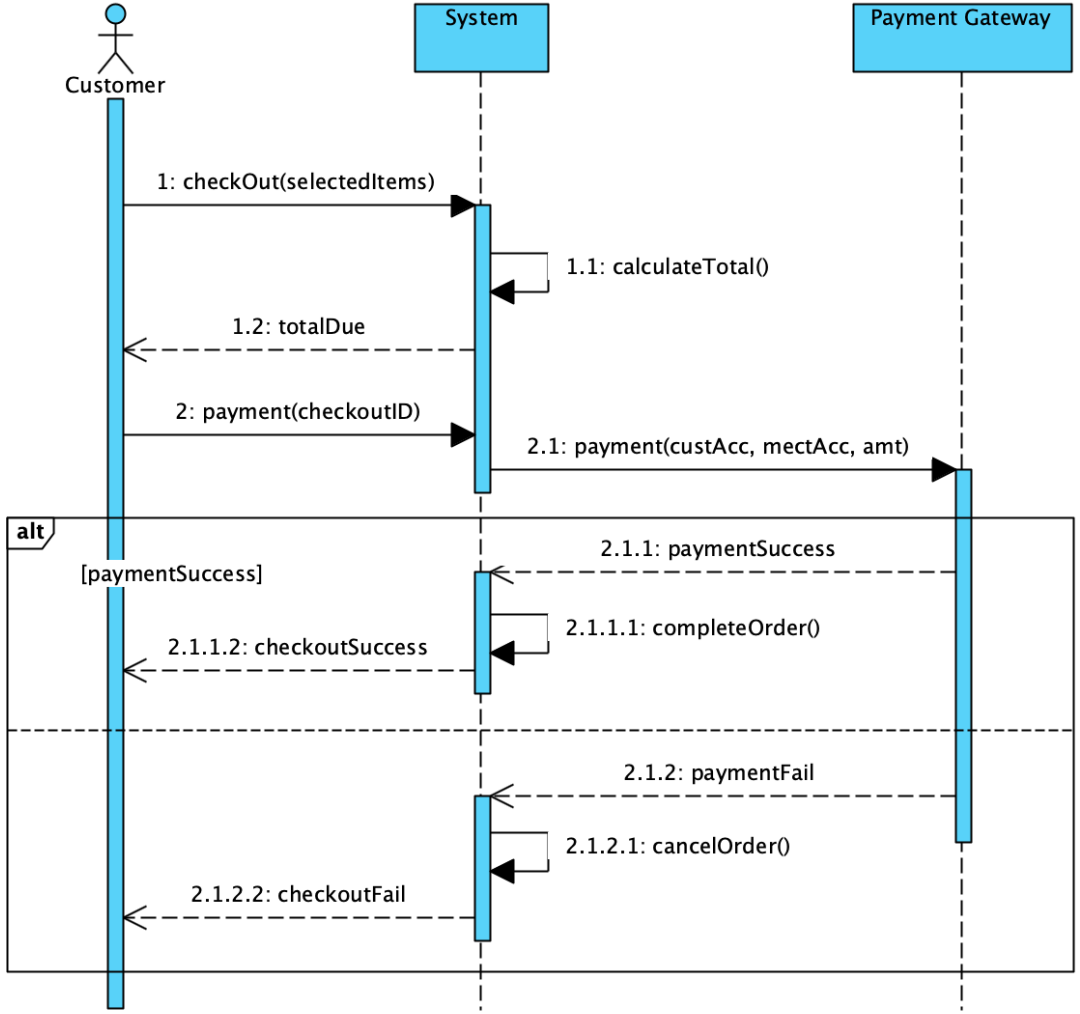
Drawing a Sequence Diagram

- Add messages in order - Start with the triggering message, then add each call and its response going down the page. For each one, decide whether it's sync or async, and label it with a method or action name plus key arguments, like save(order)
- Add activation bars - Show when each participant is busy, nesting calls inside the bar of the caller that's waiting on them.



Drawing a Sequence Diagram

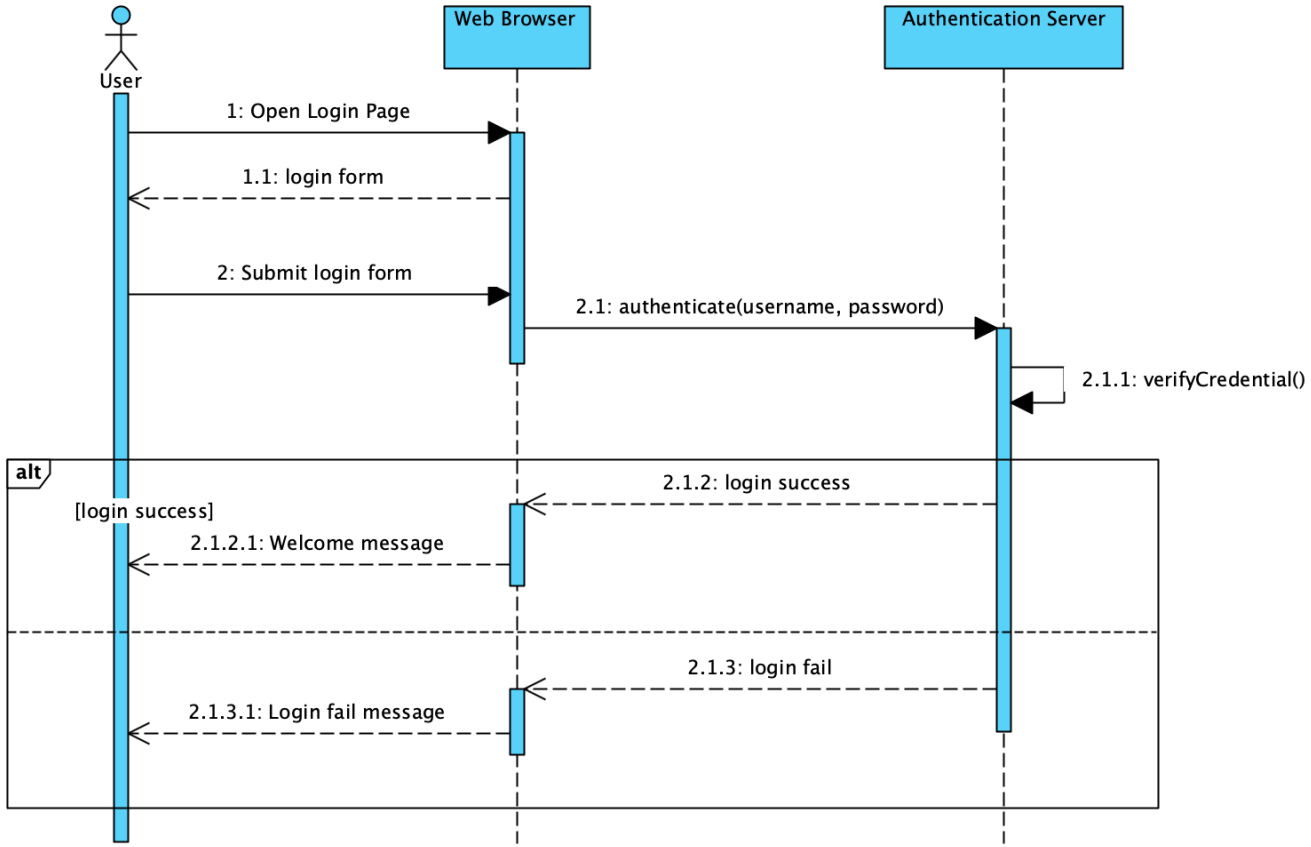
- Add logic only where it matters - Use alt, opt, or loop fragments for branches or repetition that are important to the scenario. If the fragments start piling up, split the scenario into separate diagrams instead.



Scenario — User Login

- A user opens the login page, enters a username and password, and submits the form.
- The browser sends the credentials to an authentication server, which checks them against its records.
- If valid, the server tells the browser to show a welcome message; if invalid, an error message instead.
- **Task: identify the participants, the message order, and how to represent the two outcomes.**

Worked Example — User Login



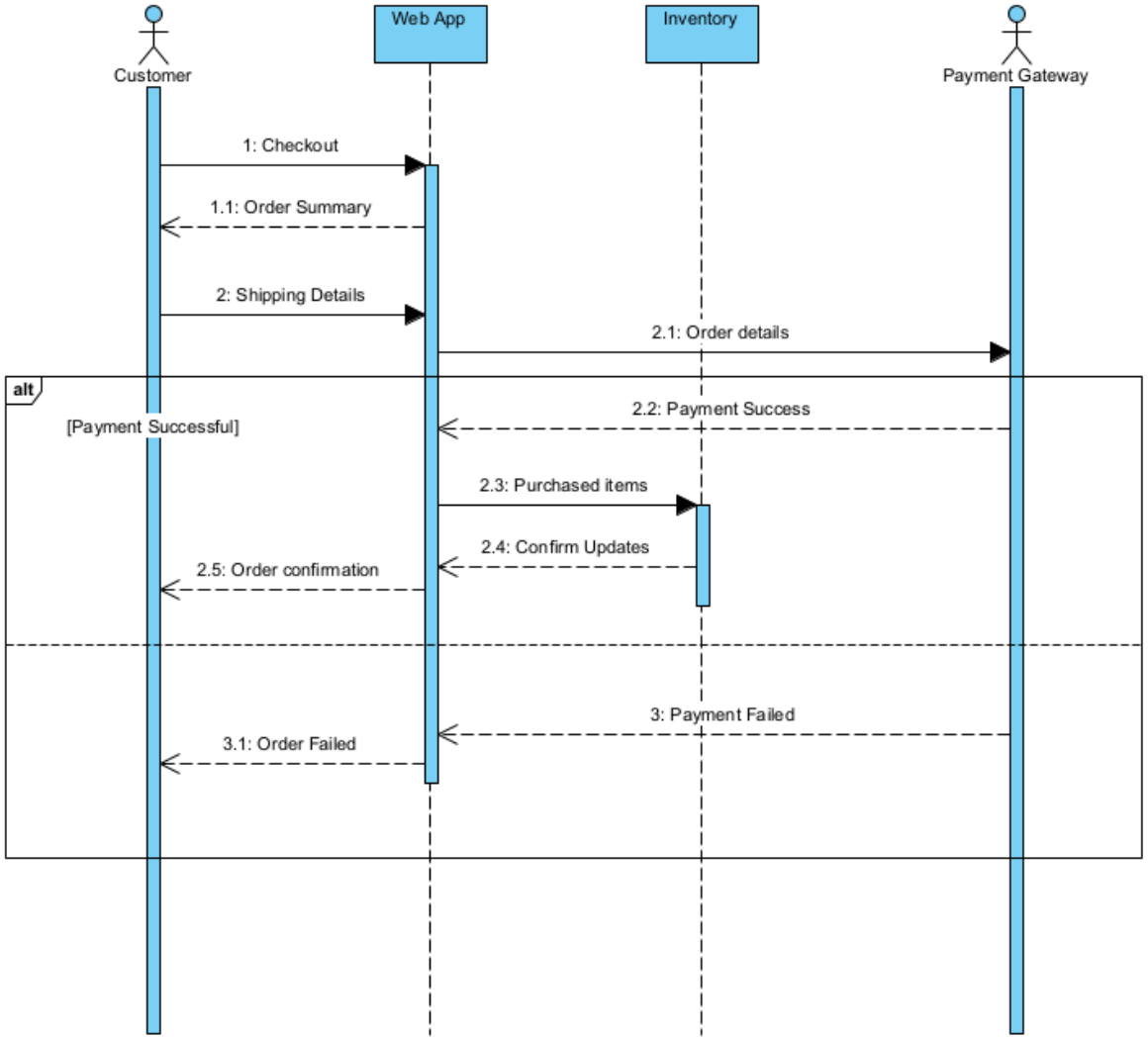
The alt frame makes the mutually exclusive outcomes of authResult() explicit.

- Browser forwards credentials to the auth server.
- Server checks them and reports success or failure.
- alt: [login successful] vs. [login failed] branches.

Scenario — Online Shopping Checkout

- A customer clicks Checkout; the web application shows an order summary and asks for shipping details.
- The application forwards the order and amount to a payment gateway, which reports back success or failure.
- Only if payment succeeded does the application update stock via an inventory system, then confirm the order.
- **Task: which combined fragment fits “only if — no explicit else”? It isn't the same one as Login.**

Worked Example — Online Shopping Checkout



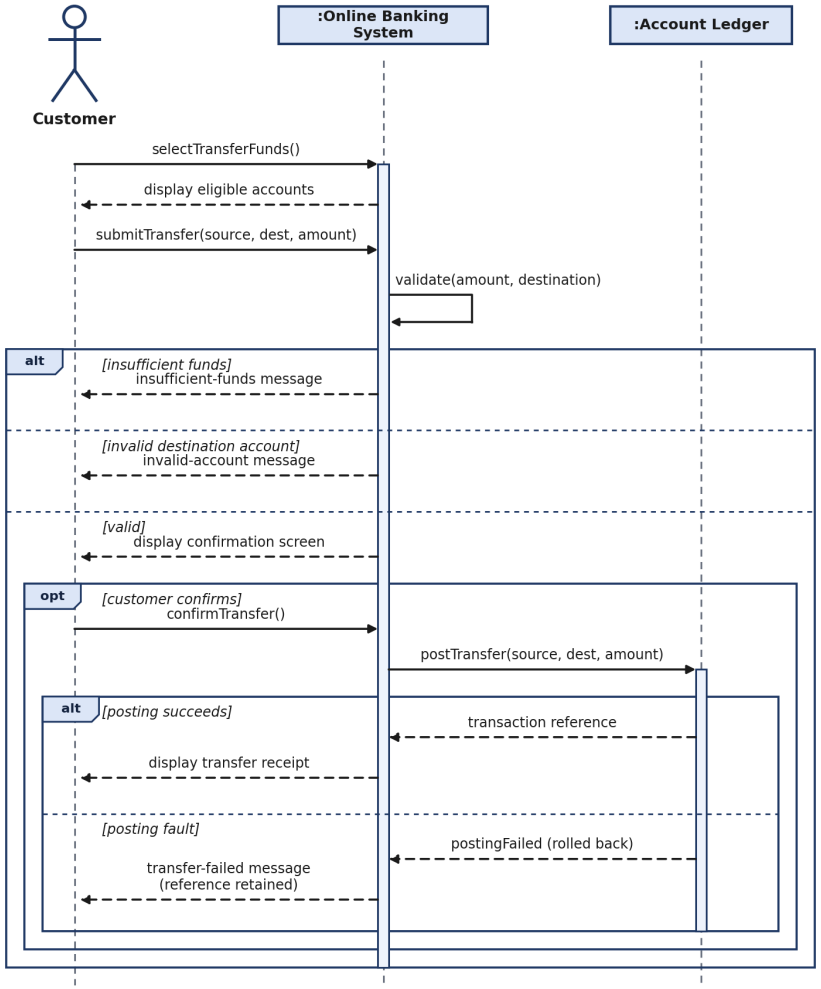
Scenario — Transfer Funds

- An authenticated customer selects Transfer Funds, picks a source/destination account and amount, and submits.
- The system validates: insufficient funds or an invalid destination account both stop the transfer with a message.
- If valid, the system shows a confirmation screen and asks the customer to confirm or cancel.
- Confirming posts the transfer via an account-ledger component; a posting fault rolls back, success shows a receipt.
- **Task: you will need more than one combined fragment — and one nested inside another.**

Fragment Escalation: alt → opt → alt

Requirement text	Becomes…	Traces to
Three mutually exclusive validation outcomes	Outer alt: [insufficient funds] / [invalid destination] / [valid].	OBS-08/09/10
Cancel → nothing further vs. confirm → post transfer, no explicit else	opt nested in [valid], guard [customer confirms].	OBS-14
Posting fails vs. posting succeeds	alt nested inside that opt.	OBS-11/13 vs. 12

Worked Example — Online Banking: Transfer Funds



- Outer alt: the three validation outcomes.
- Nested opt: customer confirms (else: nothing).
- Nested alt: posting succeeds vs. posting fault.
- Same escalation as Login → Checkout, just composed.

Questions?

Reading notes with worked examples and solved exercises are available alongside this deck.